Sport Assistance

Build a personalized AI sports coach in Python + Gradio

Student tutorial: follow the steps in order and build the project as you read.

Contents

Welcome: what you are building 3

How to use hints 3

How to use this tutorial 3

The roadmap 4

Before you start 5

Set up your project 5

Demo mode (no API key needed) 5

Testing in a Python session 5

The big picture 5

Build it, step by step 7

Step 1: Run the starter and study the worked example 7

Step 2: Feature 2, create the athlete profile 9

Step 3: Feature 1, coaching topics for each sport 11

Step 4: Feature 3, describe your playing style 13

Step 5: Feature 5, the personalized system prompt 15

Step 6: Feature 6, trace the chat, then talk to your coach 18

Step 7: Feature 4, the AI playing-style analysis prompt 21

Step 8: Feature 7, update the profile and confirm the summary 24

Step 9: Optional extensions 27

What you learned 27

Welcome: what you are building

Sport Assistance is a chat app with an AI coach that coaches **you**, not "an athlete in general". You fill in a profile (your sport, experience, dominant hand, strengths, weaknesses, preferences) and describe how you play. Then you chat with a coach that remembers all of it. By the end you will be able to:

1. Pick a sport and see coaching topics that fit it.
2. Save an **athlete profile** and update it at any time.
3. Describe your playing style and get an AI summary you can confirm or correct.
4. Chat with a coach whose answers change depending on your profile (for example, a left-handed beginner gets different advice from a right-handed advanced player).

The folder you were given is a **scaffold**. The app already launches and runs in demo mode, but the important functions are empty and marked `TODO`. This tutorial walks you through every `TODO`, in the order you should do them. **It will not hand you the code.** You write it. For each `TODO` you will find:

- **Recall**: a reminder of the Python idea you need, with a small example from a different topic (pizza, pets, scores).
- **Your task**: what the function must do, in plain English.
- **Hints**: nudges that get more specific, one at a time.
- **Check it**: how to run it and what you should see if it works.

> **TIP**
> Type everything yourself. When something breaks, read the error message slowly, because it usually tells you the file, the line number and what went wrong.

> **NOTE**
> **Instructor note:** A separate teacher answer key with full solutions is in the `answer_key/` folder. Students should not need it.

How to use hints

1. Read the **Your task** paragraph and **try it on your own for about 10 minutes**. Struggling a little is how the idea sticks.
2. Still stuck? Reveal **Hint 1** only. Try again.
3. Then **Hint 2**, then **Hint 3** (if there is one). Go in order and stop as soon as you can move on.
4. Still stuck after all the hints? Run the **Check it** test, read the error, and ask a teacher. Show them what you tried.

How to use this tutorial

- Keep this PDF open next to your editor. Work through the steps **in order**. Each step builds on the last.
- Whenever you see **Checkpoint**, stop and test before moving on.
- If you get stuck, read the **Stuck?** boxes. They list the mistakes students make most often and what the errors mean.

The roadmap

Step	File	What you build
1	`app.py`	Run the starter and study the worked example, `handle_save_profile`
2	`profile.py`	Feature 2: create the profile dictionary
3	`coach.py`	Feature 1: coaching topics for each sport
4	`profile.py`	Feature 3: store the playing-style description
5	`coach.py`	Feature 5: the personalized **system prompt** (the heart of the project)
6	`app.py` , `coach.py` , `ai_helper.py`	Feature 6: trace the chat, then try it with a real key
7	`coach.py`	Feature 4: the AI playing-style analysis prompt
8	`profile.py` , `app.py`	Feature 7: update the profile, and confirm the style summary
9	any	Optional extensions

Before you start

Set up your project

Open a terminal inside the `sport_assistance` folder and run these commands one at a time.

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env               # Windows: copy .env.example .env
python app.py
```

What each line does:

- `python -m venv .venv` creates a private "box" of Python packages just for this project.
- `source .venv/bin/activate` steps into that box. You should now see `(.venv)` at the start of your terminal line. **Do this every time you open a new terminal.**
- `pip install -r requirements.txt` installs Gradio (the web page), the Anthropic library (the AI) and python-dotenv (reads your key).
- `cp .env.example .env` makes your private settings file. You can leave the key blank for now.
- `python app.py` starts the app. Open the address it prints (usually `http://127.0.0.1:7860`).

To stop the app, click the terminal and press `Ctrl + C` . **After you edit a Python file you must stop and restart the app** to see your change.

Demo mode (no API key needed)

The AI parts need an Anthropic API key, which your teacher may give you later. **Without a key the app runs in demo mode.** The coach gives a clearly labelled canned reply instead of real advice. You can finish Steps 1 to 5 and test everything by printing prompts, so you do not need a key until Step 6.

> **WARNING**
> Never paste your API key into a Python file, share it, or commit `.env` to GitHub. Keys go only in the `.env` file. (The project's `.gitignore` already ignores it.)

Testing in a Python session

Many checkpoints ask you to try a function by itself. In the terminal (venv on, inside this folder) type `python` to open an interactive session. Lines starting with `>>>` are what you type. Type `exit()` to leave. This is faster than clicking through the app every time, and you see exactly what your function returns.

The big picture

The whole project is one idea. The AI knows **nothing** about you, so our code writes everything it needs to know into a block of text called a **prompt**:

```
form inputs ---> profile dict ---> prompt string ---> LLM ---> personalized advice
(Gradio)         (profile.py)      (coach.py)       (ai_helper.py)   (shown in Gradio)
```

1. **Profile.** The form values become a Python dictionary (`profile.py`). Gradio keeps it in a `gr.State` so it is remembered while the page is open.
2. **Prompt.** `coach.py` turns the dictionary into text: "You are a basketball coach. The athlete is left-handed, a beginner..."
3. **LLM call.** `ai_helper.py` sends that text plus the conversation to the AI.
4. **Response.** The reply appears in the chat window.

File	Job
`app.py`	The screen: the form, the tabs, the chat, and the functions that run when you click
`profile.py`	Creating and updating the profile dictionary (Features 2, 3, 7)
`coach.py`	Topics, prompt building, and demo replies (Features 1, 3, 4, 5, 6)
`ai_helper.py`	The only file that talks to the AI. **You never need to edit it.**

> **NOTE**
> `profile.py` has the same name as an old, rarely used Python library module. That is harmless here, because Python looks in your project folder first. Just do not run `python -m cProfile` in this folder.

Build it, step by step

Step 1: Run the starter and study the worked example

Goal: understand one complete, finished handler before you write your own. There is nothing to type in this step.

1a. Click around

Run `python app.py`. There are four tabs:

1. **Profile**: fill in the form and click **Save profile**. You will see a plain summary. Notice the preferences and strengths come out as `not provided`, and handedness is always `N/A`. That is because `create_profile` is a stub. Fixing it is your first job.
2. **Playing style**: two buttons that will use your profile. Right now **Confirm summary** says it is not saved.
3. **Chat with your coach**: type a message. You get a canned demo reply.
4. **Update profile**: choose a field and a new value. Nothing changes yet.

Also notice that when you change the **Sport** dropdown, the **Coaching topics** list barely changes. That is Step 3.

1b. Read `handle_save_profile`

Open `app.py` and find `handle_save_profile`. It is the pattern every other handler follows.

```
def handle_save_profile(sport, experience, handedness, strengths, weaknesses, preferences):
    # 1. Turn raw form values into a profile dictionary (your Feature 2 code).
    profile = create_profile(sport, experience, handedness, strengths, weaknesses, preferences)

    # 2. Build the text we'll show the user.
    topics = get_coaching_topics(sport)
    message = f"Profile saved for **{sport}**.\n\n" + fallback_style_summary(profile)

    # 3. Return one value per output component, in the same order as `outputs=[...]`.
    return profile, message, gr.update(choices=topics, value=topics[0])
```

Now find where it is **wired up**, further down:

```
save_btn.click(
    handle_save_profile,
    inputs=[sport, experience, handedness, strengths, weaknesses, preferences],
    outputs=[profile_state, profile_msg, topics],
)
```

Read it aloud: *"When Save is clicked, call `handle_save_profile`. Give it the current values of these six components, **in this order**. Put the three things it returns into these three components, **in this order**."*

> **CONCEPT**
> Inputs and outputs are matched by **position**, not by name. The first thing in `inputs` becomes the first parameter; the first thing returned goes into the first component in `outputs`. If a function returns three values, `outputs` must list exactly three components.

1c. What is `gr.State`?

Near the bottom of `app.py` you will find these two lines:

```
profile_state = gr.State({})   # the athlete profile dictionary
chat_state = gr.State([])      # the conversation history (list of message dicts)
```

A `gr.State` holds an ordinary Python value that Gradio **remembers between clicks** for one browser tab. Each visitor gets their own copy. Look at the handler's return line again: the profile is the first thing returned, and `profile_state` is the first output, so that is how the new profile gets **saved**. If a handler changes the profile but does not return it into the state, the change is lost.

> **WHY**
> State lives in memory only. Refresh the page and it is gone. That is fine for this project.

1d. See how the other handlers use it

Skim `handle_style_fallback`, `handle_style_ai`, `handle_chat` and `handle_update_profile`. Each one: (1) takes the saved `profile` as an input, (2) checks `if not profile:` and shows the reminder `NEED_PROFILE` if the user skipped the first tab, (3) does its job, and (4) returns the profile again.

> **CHECKPOINT**
> Can you answer these? (1) What does `gr.State` do? (2) If a handler returns `profile, message, gr.update(...)`, which three components does `outputs` need to list, and in what order? (3) Which file talks to the AI? If yes, move on.

Step 2: Feature 2, create the athlete profile

File: `profile.py` **Function:** `create_profile(sport, experience, handedness, strengths, weaknesses, preferences)`

Goal: turn the six form values into one dictionary that the rest of the app passes around.

Look at the target

The top of `profile.py` shows what a finished profile looks like. The values are different **types**: `sport` is a string, but `strengths` is a **list** of strings and `style_confirmed` is a **boolean** (`True` or `False`).

The starter version already builds the dictionary but leaves most of it empty, so the app runs:

```
profile = {
    "sport": sport,
    "experience": experience,
    "handedness": "N/A",
    "strengths": [],
    "weaknesses": [],
    "preferences": "",
    ...
}
```

Two helpers that are already written

- `side_matters(sport)` returns `True` for sports where left or right hand or foot can change advice (basketball, tennis, volleyball, soccer) and `False` for dance.
- `split_list("passing, speed , defense")` returns `["passing", "speed", "defense"]` . Gradio gives us **one string** from a textbox, but a list is easier to use in a prompt.

TODO 1: handedness, only when it matters

> **NOTE**
> **Recall:** An `if / else` runs one block when a condition is true and another when it is not, and a function that returns `True` or `False` can be used directly as the condition. A dictionary entry is changed with `d[key] = value` . For example: `if is_vegetarian(order): menu["topping"] = "mushroom"` and `else: menu["topping"] = "pepperoni"` .

Your task: Store the handedness value from the form **only for sports where it matters**. For any other sport, the profile should say `"N/A"` . (Why? The Dominant hand radio is hidden for dance, but it still has a value behind the scenes, and a dance coach should not be told "the athlete is left-handed".)

- **Hint 1:** One of the two helpers already written for you can answer the question "does it matter for this sport?"
- **Hint 2:** Use that helper as the condition of an `if` . Put the real handedness in one branch and the string `"N/A"` in the other, assigning to the `"handedness"` key of `profile` .

TODO 2: strengths and weaknesses as lists

> **NOTE**
> **Recall:** A function's answer is lost unless you **store** it. `shout = "hi".upper()` keeps the answer; a bare `"hi".upper()` throws it away. Lists live in dictionaries just like strings: `pet["toys"] = ["ball", "rope"]`.

Your task: The form gives two comma-separated strings. Make the profile's `"strengths"` and `"weaknesses"` hold **lists** instead.

- **Hint 1:** You do not have to write the splitting code, because a helper above does that job.
- **Hint 2:** Call it on the text, and assign what comes **back** into the matching dictionary key. Do this once for each of the two fields.

TODO 3: preferences

> **NOTE**
> **Recall:** `.strip()` removes spaces from both ends of a string: `" hi ".strip()` gives `"hi"`.

Your task: Store the preferences text in the profile, without stray spaces at the start or end.

- **Hint 1:** It is a string, not a list, so no splitting is needed.
- **Hint 2:** Strip the text and assign it to the `"preferences"` key.

> **CHECKPOINT**
> Run `python`, then:
> `>>> from profile import create_profile`
> `>>> print(create_profile('dance', 'Beginner', 'Left', 'turns', 'balance', ' slow music '))`
> You should see a dictionary where `handedness` is `'N/A'` (even though you passed `'Left'`, because dance is not a side-sensitive sport), `strengths` is `['turns']`, `weaknesses` is `['balance']` and `preferences` is `'slow music'` with **no extra spaces**. Repeat with `'basketball'` and `handedness` should now be `'Left'`. Then run the app, fill the form and click **Save profile**: the summary should now show your strengths, weaknesses and preferences.

> **STUCK?**
> If the summary still says `not provided`, you probably called the helper on its own line without storing the result. A function's answer disappears unless you assign it. `NameError: name 'split_list' ...` means a typo in the name. `IndentationError` means check your four-space indents.

Step 3: Feature 1, coaching topics for each sport

File: `coach.py` **Function:** `get_coaching_topics(sport)`

Goal: return a list of topics specific to the chosen sport. The Profile tab shows it in a dropdown.

Read the example

Basketball is done for you:

```
if sport == "basketball":
    return ["Shooting form", "Ball handling", "Defense", "Court vision"] + GENERAL_TOPICS
```

`GENERAL_TOPICS` is a list defined above the function. Adding two lists with `+` glues them into one longer list. That way every sport gets its own four topics plus the three general ones (warm-up, focus, consistency).

TODO 1: add `elif` branches

> **NOTE**
> **Recall:** After an `if`, you can chain more choices with `elif`, and Python runs only the **first** one that matches. For example: `if pet == "dog": return ["fetch", "walks"]` then `elif pet == "fish": return ["tank", "food"]`. Remember the colon `:` at the end of each condition line, and that `==` compares while `=` assigns.

Your task: Add one `elif` branch for each of soccer, volleyball, tennis and dance. Each should return **3 to 4 topics specific to that sport**, plus the general ones, just like the basketball example. Think like a real coach: what would a tennis coach talk about? A dancer's? The sport names must match the spelling in the `SPORTS` list at the top of `app.py` (lowercase).

- **Hint 1:** Copy the *shape* of the basketball branch, not its contents. Condition, colon, indented `return`.
- **Hint 2:** Each new branch starts with `elif sport == "..."` and returns a list of strings joined to `GENERAL_TOPICS` with `+`.
- **Hint 3:** Put all your new `elif`s **between** the basketball `if` and the final `else`.

TODO 2: keep the final `else`

Your task: Make sure the last `else` (returning just the general topics) is still there after your changes. **Think:** what would happen to an unknown sport like `"chess"` if there were no `else`? (What does a function return when it reaches the end without a `return`?)

> **CHECKPOINT**
> In `python`:
>
> ```
> >>> from coach import get_coaching_topics
> >>> print(get_coaching_topics('tennis'))
> >>> print(get_coaching_topics('chess'))
> ```
>
> The tennis list should show your tennis topics followed by `Warm-up and recovery`, `Mental focus`, `Building consistency`. The chess list should be **only** those three general topics. Then restart the app and change the **Sport** dropdown: the Coaching topics list should change for each sport. Also notice the **Dominant hand / foot** radio disappears for dance. That comes from `side_matters`.

STUCK?

`SyntaxError` on an `elif` line usually means a missing colon `:` . If a sport shows only the three general topics, the name in your `elif` does not match (check for capital letters or a typo). If you get `None` , one branch is missing its `return` .

Step 4: Feature 3, describe your playing style

File: `profile.py` **Function:** `add_style_description(profile, style_text)`

Goal: save the athlete's own description of how they play into the profile.

Why `dict(profile)`?

Look at the first line of the function:

```
updated = dict(profile)
```

`dict(profile)` makes a **copy**. If you changed `profile` directly, you would be editing the very same object Gradio has stored, which can cause confusing bugs. The pattern in this project is: *copy, change the copy, return the copy.*

TODO 1: store the text

> **NOTE**
> **Recall:** Strings have cleaning methods like `.strip()`, and you can store into a dictionary with `d["key"] = value`. For example: `pet["nickname"] = typed_name.strip()`.

Your task: Put the athlete's description, with extra spaces removed from both ends, into the `"style_text"` key of the **copy** (`updated`), not of `profile`.

- **Hint 1:** It is the same two ideas you used for `preferences` in Step 2.
- **Hint 2:** Make sure you assign to `updated[...]`, because that is the dictionary the function returns.

TODO 2: reset `style_confirmed`

> **NOTE**
> **Recall:** A boolean is the value `True` or `False`, written **without quotes**. The string `"False"` is a different thing, and (a trap!) any non-empty string counts as true in an `if`.

Your task: If the athlete edits their description, a summary they confirmed earlier might no longer be accurate. Set the `"style_confirmed"` key back to the boolean `False`.

- **Hint 1:** Same pattern as TODO 1, but the value is a boolean, not text.
- **Hint 2:** Check it has no quotation marks around it.

> **CHECKPOINT**
> Run the app. Save a profile on tab 1, go to tab 2, type a description such as `I like short passes` and click **Quick summary (no AI)**. The summary now ends with **In your own words: I like short passes**. Also try in `python`:
>
> ```
> >>> from profile import create_profile, add_style_description
> >>> p = create_profile('soccer', 'Beginner', 'Right', 'passing', '', '')
> >>> p = add_style_description(p, ' I like short passes ')
> >>> print(repr(p['style_text']), p['style_confirmed'])
> ```
>
> The text should come out with **no** spaces at its ends, and `style_confirmed` should print `False`.

STUCK?
If you get a `KeyError` or `NEED_PROFILE`, you skipped **Save profile** on tab 1. The handlers say "Please create your profile on the first tab" when the state is empty. If your text never shows up, check that you changed `updated` and not `profile`.

Step 5: Feature 5, the personalized system prompt

File: `coach.py` **Function:** `build_coaching_system_prompt(profile)`

Goal: write the instructions that tell the AI who it is and exactly who it is coaching. This is the most important function in the project. It is where "generic chatbot" becomes "your coach".

System prompt vs. messages

Every request to the AI has two parts:

- The **system prompt**: standing instructions ("You are a coach. Here is the athlete.") The user never sees it.
- The **messages**: the conversation so far, from the user and the assistant.

We rebuild the system prompt from the profile **every time** the user sends a message. That means the moment a profile changes (Feature 7), the very next answer reflects it automatically.

What is already there

```
sport = profile.get("sport", "their sport")

prompt = (
    f"You are a friendly, knowledgeable {sport} coach.\n\n"
    f"ABOUT THE ATHLETE:\n{profile_to_text(profile)}\n\n"
)
```

`profile.get("sport", "their sport")` is the safe way to read a dictionary: if the key is missing you get the fallback instead of a crash. `profile_to_text` (already written) formats the profile as bullet lines. Try printing its output if you want to see what the AI "knows".

TODO 1: get the playing style

> **NOTE**
> **Recall:** `d.get("key")` reads a dictionary value and gives `None` (no crash) if the key is missing, and an empty string, `None` and `False` all count as "false" in an `if` . For example: `if pet.get("tricks") and pet.get("trained"):` is true only when **both** have real values.

Your task: Read the athlete's playing style from the profile. **A starter quirk to notice:** `profile_to_text` already includes the user's own words (`style_text`), but it **never** includes the AI summary the athlete confirms in Step 8 (`style_summary`). Decide how to get that confirmed summary into the prompt, and make sure it is used **only when the athlete has actually confirmed one**.

- **Hint 1:** Two keys in the profile matter here: one says whether the summary is confirmed, and one holds the summary text.
- **Hint 2:** Use `.get()` on both. Combine the two checks in one `if` with `and` .
- **Hint 3:** In words: *start with an empty* `style` *variable; if the profile is confirmed and has a summary, copy that summary into* `style` . Use `style` in the next TODOs.

TODO 2 and TODO 3: personalize with `if` / `elif` , then add it to the prompt

> **NOTE**
> **Recall:** `prompt += "more text\n"` adds onto the end of an existing string, and `\n` is a new line. For example: `order += "Extra cheese.\n"` . An `elif` lets you handle a second case, and cases that match no branch can simply do nothing.

Your task: Add detail to `prompt` , one `if` at a time, so the AI is told about **this** athlete. Your prompt should include a sentence for each of these:

When...	Tell the AI to...
`style` is not empty (TODO 1)	use the athlete's confirmed style summary
handedness is `"Left"`	not assume right-handed technique, and mirror right-handed examples
handedness is `"Both"`	know both sides can be trained
experience is `"Beginner"`	use simple language and cover the basics first
the preferences text is not empty	respect those preferences

- **Hint 1:** Read each of those values out of the `profile` dictionary with `.get(...)` .
- **Hint 2:** Handedness has three cases that need different sentences, so use `if` / `elif` . For `"Right"` or `"N/A"` nothing needs adding, and that is fine.
- **Hint 3:** Experience and preferences are **separate** questions from handedness, so give each its own `if` , not another `elif` on the handedness chain. **Think:** what goes wrong if you chain them all together?
- **Hint 4:** End each sentence you add with `\n` so the lines do not run together. The word `"Left"` starts with a capital L, and `==` is case-sensitive.

TODO 4: the safety rule

Your task: Coaches must not just do whatever they are told. Write a rule for the prompt that covers all of these:

- Respect the athlete's preferences, **but**
- if a technique could be unsafe or ineffective, say so clearly, explain why, and suggest a safer alternative.
- Do not make claims about professional athletes or results the AI is not sure about.
- **Hint 1:** It is just another `prompt += ...` with a sentence or two of your own wording.
- **Hint 2:** Put it **after** the personalized lines, so it applies to everything above it.

TODO 5: the style rule

Your task: Add a short rule asking for answers that are concise and practical, and then make sure the function **returns** the finished `prompt` .

- **Hint 1:** Same `+=` pattern again. One line is plenty.

Read it as if you were the AI

Now the best test in the whole project. In `python` :

```
>>> from profile import create_profile
```

```
>>> from coach import build_coaching_system_prompt
```

```
>>> me = create_profile('basketball', 'Beginner', 'Left', 'passing, speed', 'free throws', 'short drills')
```

```
>>> print(build_coaching_system_prompt(me))
```

Read the printed prompt slowly, as if you were the AI and this were all you knew. It should contain, in this order:

- the first line "You are a friendly, knowledgeable basketball coach."
- the "ABOUT THE ATHLETE" bullet list showing basketball, Beginner, Left, passing and speed, free throws, short drills
- a sentence about the athlete being **left-handed**
- a sentence asking for **simple language** for a beginner
- a sentence about respecting the **short drills** preference
- your safety rule
- your concise-and-practical rule

Each sentence should be on its own line.

CHECKPOINT

Compare two athletes. Build a second athlete with `'Right'` and `'Advanced'` instead of `'Left'` and `'Beginner'`, and print its prompt. The left-handed and beginner lines must **disappear**. If the two prompts are identical, your `if` statements are not working. This "compare two profiles" test is the best way to check that a profile really reaches the AI. Also test a profile with `'Both'` as handedness.

STUCK?

If nothing is added for left-handed athletes, check the spelling: the form gives `"Left"` with a capital L. If you get a `KeyError`, use `profile.get("handedness")` instead of `profile["handedness"]`. If the text runs together without line breaks, add `\n` at the end of each sentence. If you get `None` printed, you forgot `return prompt`.

Step 6: Feature 6, trace the chat, then talk to your coach

Files: `app.py` (`handle_chat`), `coach.py` (`get_coaching_response`), `ai_helper.py` (`ask_llm`)

Goal: there is almost nothing to write here. The `TODO` in `handle_chat` says: *"Nothing to change here, but trace this call!"* Following a request through three files is a real programmer skill. This step teaches you how to do it, and how the AI gets its memory.

Trace 1: `handle_chat` in `app.py`

```
reply = get_coaching_response(message, chat_history, profile)

chat_history = chat_history + [
    {"role": "user", "content": message},
    {"role": "assistant", "content": reply},
]
return "", chat_history, chat_history
```

- `message` is what the user typed. `chat_history` is the earlier conversation, a list of dictionaries, each with a `"role"` (`"user"` or `"assistant"`) and `"content"` (the text). It comes out of `chat_state` .
- The handler asks `get_coaching_response` for the reply, then **adds both the user's message and the reply** to the list.
- It returns **three** values: an empty string (which clears the textbox), the history (which updates the visible chat window), and the history again (which saves it into `chat_state`).

Trace 2: `get_coaching_response` in `coach.py`

```
if not has_api_key():
    return demo_response(message, profile)

system_prompt = build_coaching_system_prompt(profile)          # who the AI is + athlete info
messages = list(history) + [{"role": "user", "content": message}]  # past + new message
return ask_llm(system_prompt, messages)
```

Two paths: no key means the canned `demo_response` . With a key, it builds the **system prompt** (your Step 5 work) and a **messages** list that is the **entire past conversation plus the new message**.

> **WHY**
> The AI has **no memory of its own**. Every request starts from nothing. If we did not resend the history, the follow-up "What about my left hand?" would be meaningless: "my left hand" compared to what? Resending the history on every message is the only reason the coach seems to remember.

Trace 3: `ask_llm` in `ai_helper.py` (read only)

This file is finished, so just read it. The key parts:

1. `has_api_key()` returns `True` only if `ANTHROPIC_API_KEY` is set (it is read from `.env` by `load_dotenv()`).
2. It trims each message down to `role` and `content` , because Gradio may attach extra fields the API does not accept.

3. `client.messages.create(model=..., system=system_prompt, messages=clean_messages)` is the actual call to the AI. Notice the `system` and `messages` are **separate** arguments, exactly as described in Step 5.
4. A `try / except` turns any failure into a readable message instead of a crash.

Optional TODO: a friendly reminder for an empty profile

> **NOTE**
> **Recall:** An empty dictionary `{}` counts as "false" in an `if`, so `if not pets:` is true when there are no pets. A function can stop early with `return`.

Your task: The comment at the top of `get_coaching_response` suggests: if the profile is empty (the user skipped the first tab), return a friendly reminder to fill in the profile first, **before** anything else happens.

- **Hint 1:** Look at how `handle_style_fallback` in `app.py` already asks the same question.
- **Hint 2:** Put your check at the very top of the function, before the `has_api_key` check, and `return` a message of your own.

See exactly what the AI would receive

You can swap the AI for a fake one that just **prints** what it was sent. Create a new file called `peek.py` next to `app.py`:

```
import coach
from profile import create_profile

def fake_ask_llm(system_prompt, messages, max_tokens=800):
    print("SYSTEM PROMPT:")
    print(system_prompt)
    print("MESSAGES SENT:")
    for m in messages:
        print("  ", m["role"], "->", m["content"])
    return "(fake reply)"

coach.ask_llm = fake_ask_llm       # swap the real LLM for our fake
coach.has_api_key = lambda: True   # pretend a key exists

me = create_profile("basketball", "Beginner", "Left", "passing, speed", "free throws", "short
    drills")
history = [
    {"role": "user", "content": "How do I improve my shot?"},
    {"role": "assistant", "content": "Start close to the hoop and bend your knees."},
]
coach.get_coaching_response("What about my left hand?", history, me)
```

Run `python peek.py`. After the system prompt from Step 5 you will see:

```
MESSAGES SENT:
   user -> How do I improve my shot?
   assistant -> Start close to the hoop and bend your knees.
   user -> What about my left hand?
```

That list is the AI's entire "memory". Three messages, and the newest one is last.

CONCEPT

This trick (replacing a function with a fake one to see what it gets) is called **mocking**, and professional programmers use it all the time to test code without calling a paid service.

Now try it for real

1. Get an API key from your teacher.
2. Open `.env` and set the key with no quotes and no spaces:

```
ANTHROPIC_API_KEY=sk-ant-your-key-here
LLM_MODEL=claude-sonnet-5-5
```

3. Stop and restart the app. The **demo mode** banner at the top should be gone.
4. Save a profile, go to the Chat tab, and ask: *How can I improve my shooting?* Then ask a follow-up like *What about my left hand?* The coach should understand the follow-up. That is the history working.

Test the personalization

- **Two profiles.** Save a left-handed beginner, ask a question, and write down the answer. Refresh the page (this clears the state), save a right-handed advanced athlete, and ask the same question. If the answers are almost identical, the profile is not reaching the prompt. Go back to Step 5.
- **An unsafe request.** Ask something like *"I want to practice with no warm-up and train through pain."* The coach should explain the risk and suggest a safer way instead of just complying. That is your safety rule working.

CHECKPOINT

Without a key the demo reply should mention your sport and handedness. With a key, a follow-up question must make sense. If you see `The AI request failed: ...` , read the end of the message: `401` or `authentication` means a wrong or misspelled key, and a `model` error means `LLM_MODEL` has a typo. The key file must be named exactly `.env` and sit next to `app.py` .

STUCK?

If the banner still shows after adding a key, you did not restart the app, or the `.env` is in the wrong folder. If the chat window shows an error about messages, make sure you did not change the two-key dictionaries in `handle_chat` (`"role"` and `"content"`).

Step 7: Feature 4, the AI playing-style analysis prompt

File: `coach.py` **Function:** `build_style_analysis_prompt(profile, include_pro_comparisons=False)`

Goal: ask the AI to summarize the athlete's playing style, **without inventing facts**, and let the user turn pro-athlete comparisons on or off.

The starter

```
prompt = (
    "Summarize this athlete's playing style in 3–5 sentences.\n\n"
    f"ATHLETE PROFILE:\n{profile_to_text(profile)}\n"
)
```

The caller is `analyze_style`, which is already written: with no key it returns the plain fallback summary; with a key it sends your prompt to the AI with the system prompt `"You are a careful sports coach who never invents facts."`

TODO 1: rules against inventing facts

> **NOTE**
> **Recall:** Language models sometimes fill gaps with confident-sounding guesses ("hallucinations"). A prompt is the place to forbid that, and you build it with `+=` just like in Step 5.

Your task: Add a short RULES section that stops the model from making things up. Your rules should cover:

- use **only** the information in the profile
- if something is unknown, **say so** instead of guessing
- do not invent facts about the athlete
- **Hint 1:** Start the section with a heading line like `RULES:` so the model can see where the rules begin, and put each rule on its own line.
- **Hint 2:** Remember the `\n` at the end of each line, and write the rules as short, direct commands.

TODO 2: two possibility lists

Your task: After the summary, ask for two short lists named **"Possible strengths"** and **"Possible tendencies"**, and tell the model to **word every item as a possibility** (using words like "may" or "seems"). Because the AI is guessing from a few sentences, its answer should *sound* like a guess.

- **Hint 1:** Name the two lists exactly, each on its own line, so the structure of the reply is predictable.
- **Hint 2:** If your instruction contains double quotes (like `"may"`), wrap the whole Python string in single quotes, or the other way around.

TODO 3: ask the athlete to confirm

Your task: End the prompt by asking the model to invite the athlete to **confirm or correct** the summary. This sets up the **Confirm summary** button you will finish in Step 8.

- **Hint 1:** One more sentence appended to `prompt`.

TODO 4: pro comparisons, on or off

NOTE

Recall: A parameter that is `True` or `False` can be used directly as an `if` condition: `if is_hungry:` (do **not** write `== "True"`). Add `else` for the other case.

Your task: The checkbox in the app passes `include_pro_comparisons` as `True` or `False`. Use `if` / `else` so that:

- **When `True`:** the prompt gets a section titled exactly `Illustrative comparisons (NOT verified matches)`, asking for one or two well-known athletes with a similar **described** style, and telling the model **not to claim statistics or facts it is not sure about**.
- **When `False`:** the prompt tells the model to skip comparisons.

Finally, return the prompt.

- **Hint 1:** The two cases need different text, which is exactly what `if` / `else` is for.
- **Hint 2:** The exact heading text in the scaffold's TODO matters, because the checkpoint below looks for it.

CONCEPT

Why the label "NOT verified matches"? Language models can state wrong facts confidently. Labeling the section keeps the user from treating a guess as truth. Good apps are honest about what the AI can and cannot know.

Check both branches

Test it in `python`. The pro section must appear **only** when the flag is `True`:

```
>>> from profile import create_profile, add_style_description
>>> from coach import build_style_analysis_prompt
>>> p = create_profile('basketball', 'Beginner', 'Left', 'passing', 'free throws', '')
>>> p = add_style_description(p, 'I am patient')
>>> print('Illustrative' in build_style_analysis_prompt(p, False))
>>> print('Illustrative' in build_style_analysis_prompt(p, True))
```

The first should print `False` and the second `True`. Then run `print(build_style_analysis_prompt(p, True))` and read the whole prompt like the AI would: does it have the rules, the two lists, the confirm request and the pro section **in a sensible order**?

CHECKPOINT

With a key, go to the **Playing style** tab, type a description, tick or untick the pro-comparison box, and click **AI style analysis**. The summary should only use details you gave it, use words like "may" and "seems", and end by asking you to confirm. If it invents things (a position or a favorite team you never mentioned), make your RULES stronger and try again. **Try to trick it** with a very short description, and see whether it makes things up. Without a key you will see the demo banner plus the plain summary, which is expected.

STUCK?
If the pro section appears even when the box is unticked, your `if` is probably checking a string instead of the boolean. If you forgot `return prompt`, the AI gets `None`.

Step 8: Feature 7, update the profile and confirm the summary

Files: `profile.py` (`update_profile`) and `app.py` (`handle_confirm_style`)

Goal: change one field of an existing profile safely, then use it to save the user's confirmed style summary. We do `update_profile` first because `handle_confirm_style` depends on it.

Part A: `update_profile(profile, field, new_value)`

The Update tab gives us a field name and a new value typed as text. The trick is that different fields hold different **types**:

Field	Type it holds	What to do with the text
`strengths` , `weaknesses`	list	turn the text into a list
`style_confirmed`	boolean	store `True` or `False` , never a string
everything else	string	store it as is

The function starts with `updated = dict(profile)` , a copy (as in Step 4). **Think:** why copy first? What could go wrong if the function changed the original dictionary instead?

TODO 3 first: ignore unknown fields

> NOTE
> **Recall:** `key in dictionary` is `True` when the key exists: `"nickname" in pet` . Code indented under an `if` only runs when the condition is true.

Your task: Everything the function does should only happen if `field` is actually a key in `updated` . For an unknown field like `"colour"` , nothing should change and the profile comes back as it was.

- **Hint 1:** Without this check, assigning to an unknown key would silently create junk keys in the profile.
- **Hint 2:** Wrap the rest of the function's logic in an `if` that uses `in` .

TODO 1: the two list fields

Your task: For `strengths` and `weaknesses` , store a **list** made from the typed text. `"defense, hustle"` must become a list of two items.

- **Hint 1:** You used a helper for exactly this in Step 2.
- **Hint 2:** The field name decides which key to change, so you can use the variable `field` as the key. To test two field names at once, check whether `field` is `in` a tuple or list of those two names.

TODO 2: everything else, and the boolean

> NOTE
> **Recall:** The string `"False"` is **truthy** (any non-empty string is). Only the real boolean `False` is false. So a boolean must be stored as a boolean, never as text.

Your task: The scaffold's hint says it stores strings, so booleans need special care. Add the extra branches so that:

- other fields are stored as plain strings
- `style_confirmed` is stored as a **real** `True` / `False`, never a string like `"True"`
- **Hint 1:** You need one more `elif` for the boolean field, placed **before** the general `else`. Order matters, because the first matching branch wins.
- **Hint 2:** Think about what a safe way to turn the incoming value into a real boolean is. The value passed in by our own code will be a real `True`, but anything typed by a user would be a string.
- **Hint 3:** One approach: only store `True` when the value **is** the real `True` (use `is`), otherwise store `False`.

CHECKPOINT

In `python`:

```
>>> from profile import create_profile, update_profile
>>> p = create_profile('basketball', 'Beginner', 'Left', 'passing', '', '')
>>> q = update_profile(p, 'strengths', 'defense, hustle')
>>> print(q['strengths'], p['strengths'])
>>> print(update_profile(p, 'colour', 'blue') == p)
>>> print(update_profile(p, 'style_confirmed', True)['style_confirmed'])
```

You should see a two-item list and the **unchanged** original (`['passing']`) on one line, so the copy worked. Then `True` (the unknown field was ignored), then `True` (the boolean was stored). In the app, go to **Update profile**, pick `preferences`, type a new value and click **Update**: you should see it in the updated summary. Then chat again: the next reply reflects the change, because the system prompt is rebuilt every time.

NOTE

Look at `EDITABLE_FIELDS` at the top of `app.py`. It contains only `preferences, strengths, weaknesses, experience, handedness`. `style_confirmed` is deliberately **not** in the dropdown, so a user typing text into the Update tab can never break the boolean. Only our own code can set it.

Part B: `handle_confirm_style(profile, edited_summary)`

What it does: on the Playing style tab, the AI summary also fills an editable box. The user can fix anything the AI got wrong, then click **Confirm summary**. Your handler saves it.

TODO 1: save the summary

NOTE

Recall: `update_profile` returns a **copy**, so you must catch the result: `profile = update_profile(profile, ...)`. If you do not assign it, the change is thrown away.

Your task: Use `update_profile` to store the (edited) summary text under the `"style_summary"` key.

- **Hint 1:** The three arguments are the profile, the field name as a string, and the new value.
- **Hint 2:** Assign the result back to `profile`.

TODO 2: mark it confirmed

Your task: Call `update_profile` again so `"style_confirmed"` becomes the real boolean `True`. This is why you made `update_profile` handle booleans in Part A.

- **Hint 1:** Same call shape as TODO 1, but a different field and a boolean value instead of text.

Bonus (do this one yourself): what should happen if the edit box is empty? Add a check at the start so the user gets a short message and the profile is left alone.

> **CHECKPOINT**
> Save a profile, type a style description, click **Quick summary (no AI)** (this is the only one that works with no key), copy some text into the "Confirm or correct" box and click **Confirm summary**. You should get a confirmation message of your own wording instead of the old TODO message. Then check it stuck: in the chat, `build_coaching_system_prompt` should now include your summary (see Step 5, TODO 1).

Close the loop

Remember Step 5, TODO 1? Your system prompt adds the summary **only** when `style_confirmed` is `True` and there is a `style_summary`. Now that both are set, the coach has a new section. Also, if the user edits their description again in Step 4's `add_style_description`, `style_confirmed` goes back to `False`, so a stale summary stops being used. All seven features now work together.

> **STUCK?**
> If Confirm always says `TODO: confirmation is not saved yet`, you are editing the wrong file or did not restart. If the confirmation does not stick, check that you reassign: `profile = update_profile(profile, ...)` (the function returns a **copy**; it does not change the original), and that you return `profile` as the first value so Gradio saves it in the state.

Step 9: Optional extensions

Finished early? Each of these is a real feature. Do them in any order. There are no hints here: you are on your own, as real programmers are.

1. **Save and load profiles.** Use `json.dump(profile, file)` and `json.load(file)` to save a profile to `profile.json` and read it back. Add a Download and an Upload button in `app.py`. (The profile only contains strings, lists and booleans, so JSON handles it perfectly.)
2. **Goal tracking.** Add a `goals` list to the profile in `create_profile`, a way to mark goals done, and a line in `profile_to_text` and the system prompt such as "Goals completed: 2 of 4". Think about which function needs to change for each piece.
3. **Training plans.** Add a "Generate a weekly plan" button that builds a prompt from the profile **and** the topic chosen in the Coaching topics dropdown (you will need to pass that dropdown as an extra input). Reuse the ideas from Steps 5 and 7.
4. **Streaming replies.** Make the text appear as it is written. The Anthropic library has `client.messages.stream(...)`, and Gradio handlers can `yield` partial results. This one touches `ai_helper.py`.
5. **A friendlier empty-state.** Show a short "what can I ask?" list in the chat using the coaching topics for the profile's sport.
6. **Swap the AI provider.** The app was designed so only `ai_helper.py` knows which AI it talks to. Change just that file to use a different provider and see that nothing else breaks.

What you learned

- How a **profile dictionary becomes a prompt**, and why that is how an AI can "know" a user.
- The Gradio pattern: **event, function, inputs, outputs**, and that `gr.State` remembers Python values between clicks.
- **System prompt vs. messages**, and why the whole chat history is resent every time.
- **Dictionaries**, safe lookups with `.get()`, copying with `dict(...)`, and checking `if key in dictionary`.
- `if / elif / else`, list concatenation, and building long strings with `+=` and f-strings.
- Handling **different data types** (string, list, boolean) and why `"False"` is not `False`.
- **Prompt safety**: rules against inventing facts, labelled guesses, and a safety rule for unsafe requests.
- How to **trace** code across files, and how to **mock** a function to see what your program really sends.